

Java LinkedList — Teaching Notes

Expanded explanations + a dry run for every code example, ready to teach from.

1. What is a LinkedList?

A `LinkedList` is a linear data structure where elements ("nodes") are not stored in contiguous memory like an array. Instead, each node stores its data plus a reference (link) to the next node (and, in Java's case, also the previous node). Java's `LinkedList` lives in `java.util` and implements both the `List` interface and the `Deque` interface.

Why does this matter? An `ArrayList` is backed by a contiguous array — great for fast indexed access, but inserting/removing in the middle means shifting every element after it. A `LinkedList` has no shifting problem: to insert a node, you just re-point a couple of references. The tradeoff is that you lose direct indexed access — to reach element 5, you must walk the chain from the start, one link at a time.

Java's built-in `LinkedList` is a **doubly linked list**: every node holds a reference to both the *next* node and the *previous* node. This lets Java walk the list forward or backward and lets Deque operations (add/remove at both ends) run in $O(1)$.

Code

```
import java.util.LinkedList;
import java.util.List;

List<Integer> numbers = new LinkedList<>();
numbers.add(10);
numbers.add(20);
numbers.add(30);
System.out.println(numbers); // [10, 20, 30]
```

Line-by-line explanation:

- `List<Integer> numbers` — declared using the interface type `List`, not `LinkedList`. Same "coding to an interface" best practice as `ArrayList`.
- `new LinkedList<>()` — creates an empty doubly linked list. Internally it just sets up two references, `first` and `last`, both pointing to `null`.
- `numbers.add(10)` — creates a new `Node` object holding 10, and links it in as the new last node.
- `System.out.println(numbers)` — `LinkedList` overrides `toString()`, walking the chain node by node to print `[10, 20, 30]`.

Dry Run

Step	Statement executed	State of 'numbers' after
1	<code>new LinkedList<>()</code>	<code>[]</code> (first = null, last = null, size = 0)
2	<code>numbers.add(10)</code>	<code>[10]</code> (first = last = Node(10))
3	<code>numbers.add(20)</code>	<code>[10, 20]</code> (Node(10) <-> Node(20))
4	<code>numbers.add(30)</code>	<code>[10, 20, 30]</code> (Node(10) <-> Node(20) <-> Node(30))
5	<code>println(numbers)</code>	Output: <code>[10, 20, 30]</code>

2. The Node — What's Actually Inside a LinkedList

This is the single most important mental model for the whole topic. Draw it on the board before anything else. Internally, Java's doubly linked list node looks like this (simplified from the real JDK source):

```
private static class Node<E> {
    E item;
    Node<E> next;
    Node<E> prev;

    Node(Node<E> prev, E element, Node<E> next) {
        this.item = element;
        this.next = next;
        this.prev = prev;
    }
}
```

Each node is its own small object floating somewhere in memory. There is no "array" underneath — the list itself just keeps two pointers, `first` and `last`. Every node knows only its immediate neighbours. To get from node A to node C, you must pass through node B — there is no shortcut.

Teaching tip: draw three boxes on the board labelled 10, 20, 30. Draw arrows next/prev between them, and draw `first` pointing at the box with 10 and `last` pointing at the box with 30. This single picture answers 90% of the questions students will ask for the rest of the class.

3. LinkedList vs ArrayList — Key Differences

This comparison is a classic interview question, and is the natural sequel to the `ArrayList` class. Teach it as: "ArrayList is fast to read, slow to insert/delete in the middle; LinkedList is the opposite — fast to insert/delete once you're there, slow to reach a position."

Feature	ArrayList	LinkedList
Underlying structure	Resizable array (contiguous memory)	Chain of Node objects (scattered memory)
Random access <code>get(i)</code>	$O(1)$ — direct index	$O(n)$ — must walk from an end
Insert/remove at start	$O(n)$ — shifts everything	$O(1)$ — just re-link <code>first</code>
Insert/remove at end	Amortized $O(1)$	$O(1)$ — <code>last</code> is tracked
Insert/remove in middle	$O(n)$ — shifts elements	$O(n)$ to find the spot, but $O(1)$ to actually link once there
Memory overhead	Low — just the values (+ unused capacity)	Higher — each node stores 2 extra references (next, prev)
Implements	List	List and Deque (so it can act as a stack, queue, or double-ended queue)
Cache performance	Good — contiguous memory is CPU-cache-friendly	Poor — nodes are scattered, causing more cache misses

Teaching tip: ask the class "if you're building a music playlist where songs get added/removed from the middle constantly, but you rarely jump to song #47 directly, which is better?" Answer: `LinkedList`. "If you're storing exam scores and constantly doing `scores.get(23)` ?" Answer: `ArrayList`.

4. Singly vs Doubly Linked Lists

Before diving further into Java's class, it helps to separate the general data-structure concept from Java's specific implementation.

- **Singly linked list** — each node only has a `next` reference. You can only travel forward. To find the node before a given node, you must start over from the head.
- **Doubly linked list** — each node has both `next` and `prev`. You can travel in either direction, and deleting a node you already have a reference to is $O(1)$, because you don't need to search for its predecessor.

Java's `java.util.LinkedList` is always doubly linked. There is no built-in singly linked list class in Java — if a course asks students to implement one, they build it manually as a custom class (see Section 12).

5. Declaring, Creating & Generics

```
// Declare with the interface type (best practice)
List<String> names = new LinkedList<>();

// Declare with the Deque interface when you need stack/queue behaviour
Deque<String> deque = new LinkedList<>();

// From an existing collection
List<String> copy = new LinkedList<>(names);
```

Explanation of each line:

- `List<String> names = new LinkedList<>()` — coding to the `List` interface. If a student later realizes they need Deque-style operations, this line has to change; that's normal and expected.
- `Deque<String> deque = new LinkedList<>()` — `LinkedList` also implements `Deque`, so it can be declared as a double-ended queue when the intent is to use it as a stack or queue rather than a general list.
- `new LinkedList<>(names)` — copy constructor. Walks through `names` and creates a brand-new independent chain of

nodes holding the same elements.

Unlike `ArrayList`, there is no capacity-hint constructor for `LinkedList` — there is no backing array to size up front, so that concept doesn't apply here at all. This is a good contrast point to raise explicitly.

6. Internal Working — How `add()`, `addFirst()`, `addLast()` Actually Link Nodes

This is the concept interviewers love to probe for `LinkedList`, the same way resizing is probed for `ArrayList`. There is no array to grow — instead, "internal working" means understanding exactly which references get updated.

`addLast(x)` — adding to the end

```
void linkLast(E e) {
    Node<E> oldLast = last;
    Node<E> newNode = new Node<>(oldLast, e, null);
    last = newNode;
    if (oldLast == null) first = newNode; // list was empty
    else oldLast.next = newNode;
    size++;
}
```

Walk the class through it out loud: "create a brand-new node whose `prev` points at the current last node and whose `next` is null. Make this new node the list's `last`. Then, tell the OLD last node that its `next` is now this new node. No shifting, no copying — just re-pointing four references."

Dry Run — building [10, 20, 30] via `addLast` three times

Step	Call	first	last	Links
1	<code>addLast(10)</code>	<code>Node(10)</code>	<code>Node(10)</code>	<code>Node(10).next=null, .prev=null</code>
2	<code>addLast(20)</code>	<code>Node(10)</code>	<code>Node(20)</code>	<code>Node(10).next=Node(20), Node(20).prev=Node(10)</code>
3	<code>addLast(30)</code>	<code>Node(10)</code>	<code>Node(30)</code>	<code>Node(20).next=Node(30), Node(30).prev=Node(20)</code>

`addFirst(x)` — adding to the start (mirror image, also $O(1)$)

```
void linkFirst(E e) {
    Node<E> oldFirst = first;
    Node<E> newNode = new Node<>(null, e, oldFirst);
    first = newNode;
    if (oldFirst == null) last = newNode; // list was empty
    else oldFirst.prev = newNode;
    size++;
}
```

This is exactly why `LinkedList` shines at the front: `ArrayList.add(0, x)` must shift every single element one slot to the right ($O(n)$), but `LinkedList.addFirst(x)` only touches two references ($O(1)$).

★ **IMPORTANT — TEACH THIS:** `LinkedList` never resizes and never shifts elements. Every insertion is "create a node, re-point a few references." Contrast this directly against `ArrayList`'s resize-and-copy behaviour from the previous class — this contrast is the #1 follow-up interview question.

7. Core Methods (add, get, set, remove, and Deque-style methods)

Method	Purpose	Example	Cost
<code>add(x)</code> / <code>addLast(x)</code>	append to end	<code>list.add(10)</code>	$O(1)$
<code>addFirst(x)</code>	insert at front	<code>list.addFirst(5)</code>	$O(1)$
<code>add(i, x)</code>	insert at index i	<code>list.add(2, 99)</code>	$O(n)$ — must walk to index i first
<code>get(i)</code>	read element at index i	<code>list.get(0)</code>	$O(n)$
<code>getFirst()</code> / <code>getLast()</code>	peek at either end	<code>list.getFirst()</code>	$O(1)$
<code>set(i, x)</code>	overwrite element at index i	<code>list.set(0, 5)</code>	$O(n)$ to find + $O(1)$ to overwrite
<code>remove(i)</code>	remove by index	<code>list.remove(0)</code>	$O(n)$
<code>removeFirst()</code> / <code>removeLast()</code>	remove from either end	<code>list.removeFirst()</code>	$O(1)$

remove(Object)	remove first matching value	list.remove(Integer.valueOf(5))	O(n)
size()	number of elements	list.size()	O(1) — tracked as a field, not counted each time
contains(x)	membership check	list.contains(5)	O(n)
push(x) / pop()	stack behaviour (LIFO)	list.push(1)	O(1)
offer(x) / poll()	queue behaviour (FIFO)	list.offer(1)	O(1)

The `push/pop/offer/poll` family exists because `LinkedList` implements `Deque`. This is worth calling out explicitly: **the same `LinkedList` object can behave as a stack, a queue, or a general list**, just by which methods you choose to call on it.

The classic pitfall: `remove(int)` vs `remove(Object)` — same trap as `ArrayList`

```
List<Integer> list = new LinkedList<>(List.of(10, 20, 30));
list.remove(1);           // removes INDEX 1 -> [10, 30]
list.remove(Integer.valueOf(10)); // removes VALUE 10 -> [30]
```

Exactly the same overload-resolution trap that exists in `ArrayList` exists here too, because both classes implement the same `List` interface with the same two `remove` signatures. If this class follows the `ArrayList` class, this is a good five-second callback: "remember this trap from last time? It's identical here."

Dry Run

Step	Statement	list contents	Explanation
1	<code>new LinkedList<>(List.of(10,20,30))</code>	[10, 20, 30]	initial list
2	<code>list.remove(1)</code>	[10, 30]	int argument → walks to index 1, unlinks that node (was 20)
3	<code>list.remove(Integer.valueOf(10))</code>	[30]	Object argument → searches for and unlinks the node holding value 10

8. Iterating a `LinkedList`

```
// 1) for-each (cleanest for read-only forward traversal)
for (int v : list) {
    System.out.println(v);
}

// 2) ListIterator (can move forward AND backward, and supports safe removal/insertion)
ListIterator<Integer> it = list.listIterator();
while (it.hasNext()) {
    int v = it.next();
    if (v % 2 == 0) it.remove(); // SAFE removal
}

// 3) Descending iterator (built-in, walks from last to first)
Iterator<Integer> dit = list.descendingIterator();
while (dit.hasNext()) {
    System.out.println(dit.next());
}
```

Explanation of each style:

- **for-each** — internally uses an iterator that follows `next` references one at a time. Simple and readable, but cannot safely add/remove mid-loop, same rule as `ArrayList`.
- **ListIterator** — a more powerful iterator than the plain `Iterator` `ArrayList` used. It can move both directions (`hasNext/next` and `hasPrevious/previous`) and can safely `add()` during traversal, not just `remove()`.
- **descendingIterator()** — a convenience method unique to `LinkedList` (via `Deque`) that walks the chain backward from `last` to `first`, taking advantage of the `prev` links.

★ **IMPORTANT — TEACH THIS:** Using `list.get(i)` inside a loop (`for (int i = 0; i < list.size(); i++) list.get(i)`) on a `LinkedList` is a common and serious beginner mistake — each `get(i)` re-walks the chain from an end, turning an O(n) traversal into an accidental O(n²)! Always use `for-each` or an iterator on a `LinkedList` instead of index-based access.

Dry Run — `ListIterator` removing even numbers from [10, 15, 22, 7, 8]

Call <code>v = it.next()</code>	<code>v % 2 == 0?</code>	Action	list after this step
---------------------------------	--------------------------	--------	----------------------

10	true	it.remove()	[15, 22, 7, 8]
15	false	keep	[15, 22, 7, 8]
22	true	it.remove()	[15, 7, 8]
7	false	keep	[15, 7, 8]
8	true	it.remove()	[15, 7]

Final result: [15, 7]. Just like with ArrayList's Iterator, this avoids `ConcurrentModificationException` because the removal happens through the iterator itself, which keeps its internal state in sync.

9. LinkedList as a Stack and a Queue (Deque Behaviour)

This is the section that makes LinkedList feel genuinely different from ArrayList, and is worth spending real time on because it directly sets up later topics like stacks, queues, and BFS traversal in trees/graphs.

Role	Method to add	Method to remove	Order
Stack (LIFO)	push(x)	pop()	Last In, First Out
Queue (FIFO)	offer(x) / add(x)	poll() / remove()	First In, First Out
Deque (both ends)	addFirst(x) / addLast(x)	removeFirst() / removeLast()	Either end, your choice

```
Deque<Integer> stack = new LinkedList<>();
stack.push(1);
stack.push(2);
stack.push(3);
System.out.println(stack.pop()); // 3 (last one pushed comes out first)

Deque<Integer> queue = new LinkedList<>();
queue.offer(1);
queue.offer(2);
queue.offer(3);
System.out.println(queue.poll()); // 1 (first one offered comes out first)
```

Dry Run — stack.push(1), push(2), push(3), then pop()

Step	Call	Stack contents (top → bottom)
1	push(1)	[1]
2	push(2)	[2, 1]
3	push(3)	[3, 2, 1]
4	pop()	returns 3 → remaining: [2, 1]

Teaching tip: push()/pop() work at the *front* of the LinkedList internally (addFirst/removeFirst) — both O(1), which is exactly why LinkedList makes an efficient stack. Ask students: "why not implement a stack using ArrayList by pushing/popping from index 0?" Answer: that would be O(n) every time, because of the shifting problem from the previous class.

10. Searching in a LinkedList

Because there's no random access, searching a LinkedList only has one real strategy: linear scan from one end.

```
FUNCTION linearSearch(list, target):
    node = list.first
    index = 0
    WHILE node != null:
        IF node.item == target: RETURN index
        node = node.next
        index = index + 1
    RETURN -1

// Or simply: list.indexOf(target) / list.contains(target)
```

`indexOf` and `contains` do exactly this internally — they cannot skip ahead the way binary search can on a sorted array, because there is no way to "jump" to the middle node without first walking there. This is why `Collections.binarySearch()` on a LinkedList is technically legal but practically pointless — it still degrades toward O(n) because getting to the middle costs

O(n) each time.

Dry Run — linearSearch([4, 9, 2, 7, 5], target = 7)

index	node.item	Match target 7?	Action
0	4	No	move to node.next
1	9	No	move to node.next
2	2	No	move to node.next
3	7	Yes	RETURN 3

11. Sorting a LinkedList

```
List<Integer> nums = new LinkedList<>(List.of(5, 3, 8, 1, 9));
Collections.sort(nums); // ascending, natural order
nums.sort(Comparator.reverseOrder()); // descending
```

`Collections.sort()` and `list.sort()` work on `LinkedList` exactly as they do on `ArrayList`, because both come from the shared `List` interface. Under the hood, `Collections.sort` actually copies the elements into an array, sorts that array with `TimSort`, then walks the list once more relinking nodes in the new order — because sorting the chain in place by swapping links repeatedly would be much slower and harder to get right than sorting an array.

Dry Run — Collections.sort([5, 3, 8, 1, 9])

Stage	State
Start	[5, 3, 8, 1, 9]
Elements copied to a temporary array and sorted there	[1, 3, 5, 8, 9]
Nodes re-linked in the new order	[1, 3, 5, 8, 9]

12. Custom Objects in a LinkedList

```
class Student {
    String name;
    int marks;
    Student(String name, int marks) { this.name = name; this.marks = marks; }
    public String toString() { return name + ":" + marks; }
}

LinkedList<Student> students = new LinkedList<>();
students.addLast(new Student("Riya", 88));
students.addFirst(new Student("Aman", 72));
System.out.println(students); // [Aman:72, Riya:88]
```

Notice `addFirst` is used here instead of a plain `add` to deliberately show that ordering in a `LinkedList` is about explicit front/back placement, not just "always goes to the end" the way plain `add()` on `ArrayList` feels.

Dry Run

Step	Action	students list
1	students.addLast(new Student("Riya", 88))	[Riya:88]
2	students.addFirst(new Student("Aman", 72))	[Aman:72, Riya:88]

13. Building a Simple Singly Linked List by Hand (Conceptual Exercise)

Even though Java's built-in class is doubly linked, it is extremely valuable to have students hand-build a tiny singly linked list once, so the "node and pointer" idea stops being abstract. This is a good whiteboard-and-code exercise, not something to rush.

```
class Node {
    int data;
    Node next;
    Node(int data) { this.data = data; }
}

class SimpleLinkedList {
```

```

Node head;

void addLast(int value) {
    Node newNode = new Node(value);
    if (head == null) { head = newNode; return; }
    Node curr = head;
    while (curr.next != null) curr = curr.next;    // walk to the last node
    curr.next = newNode;
}

void print() {
    Node curr = head;
    while (curr != null) {
        System.out.print(curr.data + " -> ");
        curr = curr.next;
    }
    System.out.println("null");
}
}

```

Dry Run — addLast(10), addLast(20), addLast(30)

Step	Call	head	Chain
1	addLast(10)	Node(10)	10 → null
2	addLast(20)	Node(10)	10 → 20 → null (walked from head to find the last node, then linked)
3	addLast(30)	Node(10)	10 → 20 → 30 → null

Pitfall to flag for students: this hand-written `addLast` is $O(n)$ because it walks the whole chain every single call — there's no tracked `tail` reference. Ask the class how they'd fix this (answer: keep a `tail` field, exactly like Java's real `LinkedList` keeps `last`).

14. Time Complexity Cheat-Sheet

Operation	Time
<code>addFirst(x)</code> / <code>addLast(x)</code>	$O(1)$
<code>removeFirst()</code> / <code>removeLast()</code>	$O(1)$
<code>get(i)</code> / <code>set(i, x)</code>	$O(n)$
<code>add(i, x)</code> / <code>remove(i)</code>	$O(n)$ to reach index i , $O(1)$ to link once there
<code>contains(x)</code> / <code>indexOf(x)</code>	$O(n)$
<code>size()</code>	$O(1)$ (tracked as a field)
<code>push()</code> / <code>pop()</code> / <code>offer()</code> / <code>poll()</code>	$O(1)$
<code>Collections.sort</code> / <code>list.sort</code>	$O(n \log n)$

★ **IMPORTANT — TEACH THIS:** Make students directly contrast this table against the `ArrayList` cheat-sheet from the previous class, side by side. That contrast — $O(1)$ at the ends vs $O(n)$ for random access, and vice versa for `ArrayList` — is the single most-asked interview question on this whole topic.

15. Common Mistakes & Interview Tips

- Using `get(i)` in a loop on a `LinkedList` — turns an $O(n)$ traversal into $O(n^2)$. Always prefer `for-each` or an iterator (Section 8).
- Confusing `remove(int index)` with `remove(Object o)` — identical trap to `ArrayList` (Section 7).
- Forgetting that `LinkedList` implements `Deque` — students often reach for a separate `Stack` or `Queue` class when a `LinkedList` already does the job via `push/pop/offer/poll`.
- Assuming `LinkedList` is "generally faster" than `ArrayList` — it is only faster for insert/delete at known positions (especially the ends). For read-heavy code it is usually slower, due to both $O(n)$ traversal and worse CPU cache locality.
- Modifying a list while iterating with `for-each` → use `ListIterator.remove()` / `add()` instead, same rule as `ArrayList`.
- Forgetting each node carries extra memory overhead (two references per element) — for a list of a million small integers, this overhead can matter.
- Always state Big-O for the exact `LinkedList` operation used when explaining a solution out loud — " $O(1)$ because I used

`addFirst`, not `add(0, x)`" shows real understanding.

End of expanded notes — every code block above now has a full explanation and a step-by-step dry run to teach from directly.